6061 | Introduction to Deep Learning

Exercise 3

Prof. Dr. Florian Lemmerich[1] Andreas Einwiller[1] Max Klabunde[1*]

[1]Professorship for Applied Machine Learning
University of Passau

[*](2021–2025) Preparation, creation, and presentation

29.10.2026

Goals

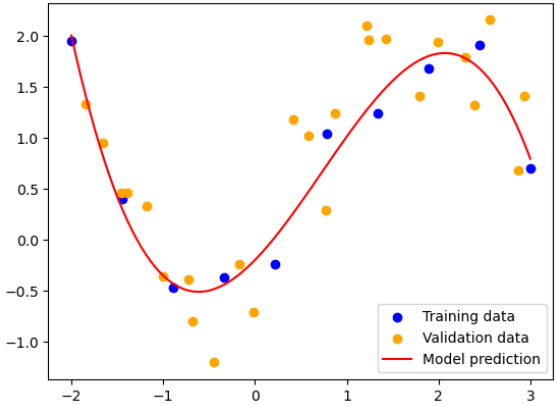
You can

- explain how MSE and MAE differ
- manually evaluate a regression model with MSE and MAE
- manually evaluate binary and multi-class classification models
- manually apply gradient descent to minimize functions

Section 1

Evaluation

Evaluation of regression



- Want to compare predictions with the ground truth
- For continuous outputs, e.g., mean absolute error (MAE) or mean squared error (MSE)
- Difference?

Example regression evaluation

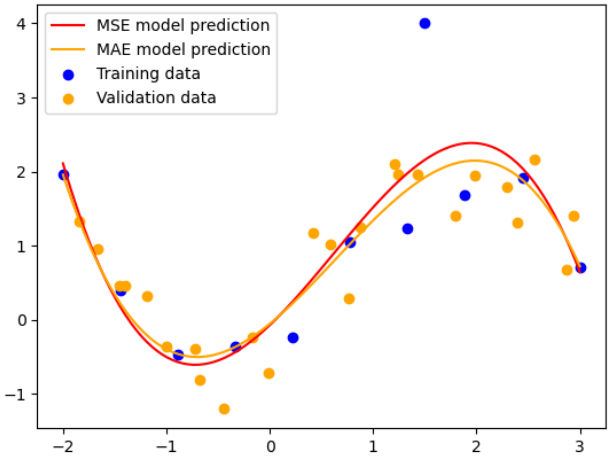
Calculate MAE and MSE for the following data

$\hat{y}$	y	$\lvert\hat{y}-y\rvert$	$(\hat{y}-y)^2$
5	4	1	1
7	9	2	4
9	8	1	1
3	2	1	1
100	12	88	7744

$$MAE = \frac{1}{N}\sum_{i=1}^{N}|\hat{y}_i - y_i| = \frac{93}{5}$$

$$MSE = \frac{1}{N}\sum_{i=1}^{N}|(\hat{y}_i - y_i)^2 = \frac{7744+\epsilon}{5}$$

Fitting a model with MAE vs. MSE



Evaluation of (binary) classification

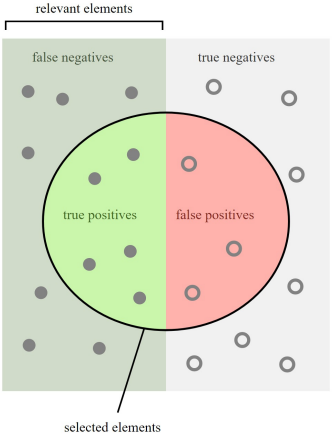


Image credit: Wikipedia

Evaluation of (binary) classification

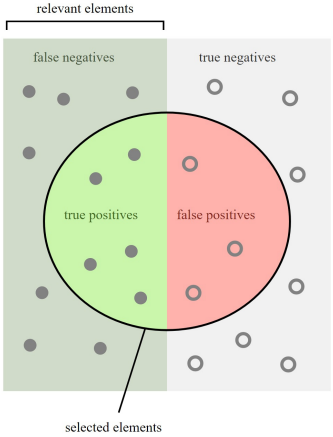


Image credit: Wikipedia

$\hat{y}$	y	Type of prediction
0	0	
0	1	
1	0	
1	1	

Evaluation of (binary) classification

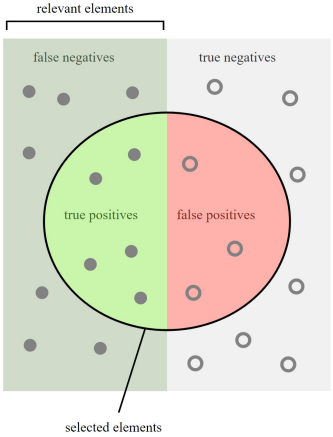


Image credit: Wikipedia

$\hat{y}$	y	Type of prediction
0	0	TN
0	1	
1	0	
1	1	

Evaluation of (binary) classification

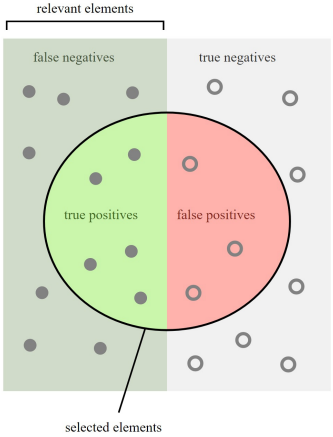


Image credit: Wikipedia

$\hat{y}$	y	Type of prediction
0	0	TN
0	1	FN
1	0	
1	1	

Evaluation of (binary) classification

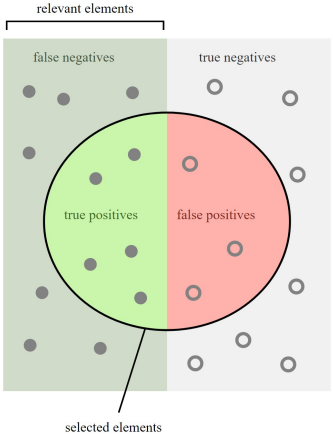


Image credit: Wikipedia

$\hat{y}$	y	Type of prediction
0	0	TN
0	1	FN
1	0	FP
1	1	

Evaluation of (binary) classification

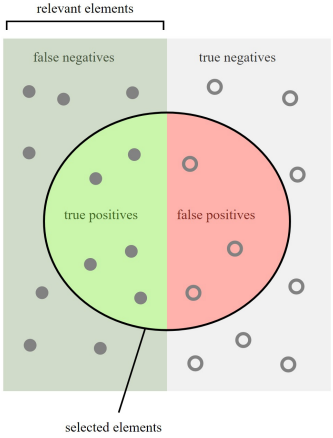


Image credit: Wikipedia

$\hat{y}$	y	Type of prediction
0	0	TN
0	1	FN
1	0	FP
1	1	TP

Evaluation of classification

- Precision $= \frac{\mathrm{TP}}{\mathrm{TP}+\mathrm{FP}}$
- Recall $= \frac{\mathrm{TP}}{\mathrm{TP}+\mathrm{FN}}$
- Accuracy $= \frac{\mathrm{TP}+\mathrm{TN}}{\mathrm{TP}+\mathrm{FN}+\mathrm{TN}+\mathrm{FP}}$
- F1-Score $= 2 \cdot \frac{\mathrm{Precision} \cdot \mathrm{Recall}}{\mathrm{Precision}+\mathrm{Recall}}$

Example predictions

		Prediction	
		positive	negative
Ground truth	positive	8	2
	negative	16	974

Results

- Precision $= \frac{8}{8+16} = 1/3$
- Recall $= \frac{8}{8+2} = 4/5$
- Accuracy $= \frac{8+974}{1000} = 0.982$
- F1-Score $= 2 \cdot \frac{1/3 \cdot 4/5}{1/3+4/5} = 8/17 \approx 0.47$

More classes

		Predicted		
		$\widehat{Y} = 0$	$\widehat{Y} = 1$	$\widehat{Y} = 2$
Ground truth	$Y = 0$	3	147	33
	$Y = 1$	17	833	137
	$Y = 2$	86	84	160

One-class view

Treat class 0 as the positive class and all others as negative.

		Predicted	
		$\widehat{Y} = 0$	Others
Ground truth	$Y = 0$	3	147+33
	Others	17+86	(833+137)+(84+160)

Results

Per class:

$$\text{Precision}_{Y=0} = \frac{3}{3+17+86} \approx 0.028$$

$$\text{Precision}_{Y=1} = \frac{833}{147+833+84} \approx 0.78$$

$$\text{Precision}_{Y=2} = \frac{160}{33+137+160} \approx 0.48$$

Results, cont

$$\text{Recall}_{Y=0} = \frac{3}{3 + 147 + 33} \approx 0.016$$

$$\text{Recall}_{Y=1} = \frac{833}{17 + 833 + 137} \approx 0.84$$

$$\text{Recall}_{Y=2} = \frac{160}{86 + 84 + 160} \approx 0.48$$

Results, cont

$$\text{macro-F1} = \frac{1}{3}\sum_{i=0}^{2} \text{F1-score}_{Y=i} \approx 0.44$$

$$\text{Accuracy} = \frac{3 + 833 + 160}{1500} = 0.664$$

Evaluation in PyTorch

https://lightning.ai/docs/torchmetrics/stable/

```
cd your-project-dir && uv add torchmetrics
```

```python
import torch
# Import eval library
import torchmetrics

# Simulate a classification problem
preds = torch.randn(10, 5).softmax(dim=-1)
target = torch.randint(5, (10,))

acc = torchmetrics.functional.accuracy(preds, target, task="multiclass",
    num_classes=5)
```

Class-based metric for accumulation over multiple steps (and multi-device training)

```python
# Initialize metric
metric = torchmetrics.classification.Accuracy(task="multiclass",
    num_classes=5)

n_batches = 10
for i in range(n_batches):
    # Simulate a classification problem
    preds = torch.randn(10, 5).softmax(dim=-1)
    target = torch.randint(5, (10,))
    # Metric on current batch
    acc = metric(preds, target)
    print(f"Accuracy on batch {i}: {acc}")

# Metric over all batches (accumulated internally)
acc = metric.compute()
print(f"Accuracy on all data: {acc}")

# Reset the internal state such that metric is ready for new data
metric.reset()
```

Your Tasks

Calculate MSE and MAE:

$\hat{y}$	y	$\lvert\hat{y} - y\rvert$	$(\hat{y} - y)^2$
.4	.5		
.7	1.7		
.6	.5		
.2	.3		
.1	.1		

Calculate precision, recall, accuracy, and F1-score:

		Predicted $\hat{y}$	
		1	0
Ground truth y	1	5	17
	0	1	53

Section 2

Gradient Descent

Training models

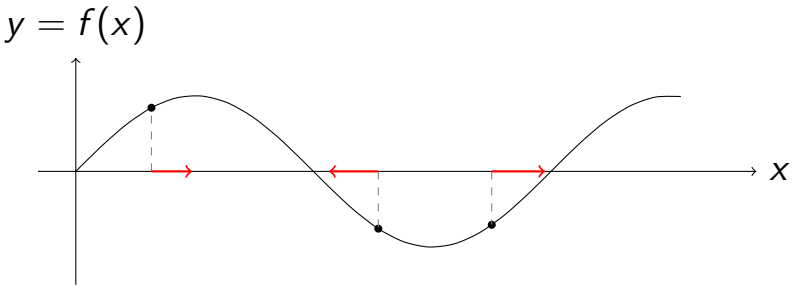
- Model f should perform well
- Perform well = low error
- Error is defined via a function E that depends on ground truth (y_i) and predictions $(\hat{y}_i)$
- Prediction is dependent on input data (x_i) and parameters (e.g., w)

Optimization problem:

$$w' = \min_w \frac{1}{n} \sum_i^n E(f(x_i; w), y_i), \text{where } f(x_i; w) = \hat{y}_i.$$

Gradients point to nearest maximum

Observe that the gradient points in the direction of the steepest local ascent



Iterative optimization with Gradient Descent

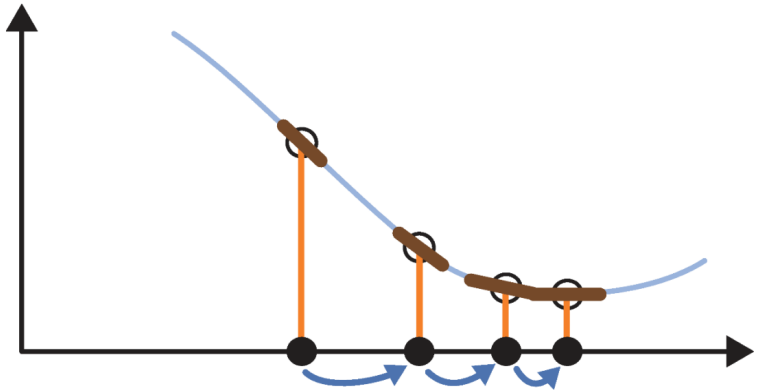
UNIVERSITY OF PASSAU

1. Initialize parameters w randomly
2. Compute $\frac{\partial E}{\partial w} = \begin{bmatrix} \frac{\partial E}{\partial w_0} \\ \vdots \\ \frac{\partial E}{\partial w_k} \end{bmatrix}$
3. Update parameters $w_i^{t+1} \leftarrow w_i^t - \alpha \frac{\partial E}{\partial w_i^t}$, where $i \in [0, k]$

Repeat steps 2 and 3 until convergence or some limit.
α (the learning rate) is a small positive number (e.g., $\alpha = 0.001$)

1D illustration

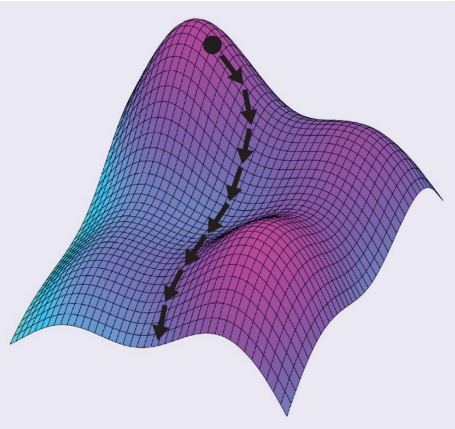
x-axis: w; y-axis: $E(w)$



A. Glassner: Deep Learning - A visual approach

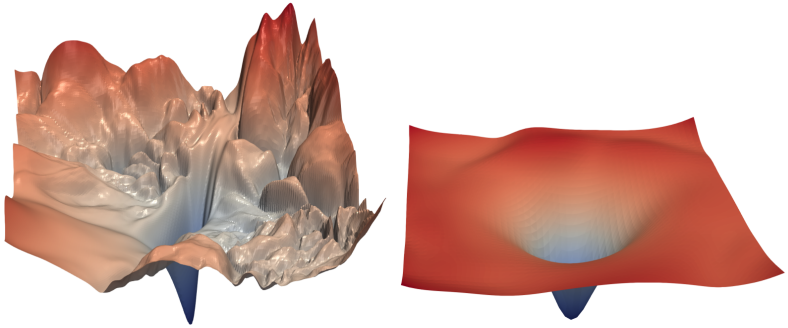
2D illustration

x,y-axis: w_0, w_1; z-axis: $E(w)$



A. Glassner, Deep Learning: A visual approach

Issue: many saddle points, sharp minima, ...



(a) without skip connections (b) with skip connections

Figure 1: The loss surfaces of ResNet-56 with/without skip connections. The proposed filter normalization scheme is used to enable comparisons of sharpness/flatness between the two figures.

Li et al., 2017. Visualizing the Loss Landscape of Neural Nets.

Example 1

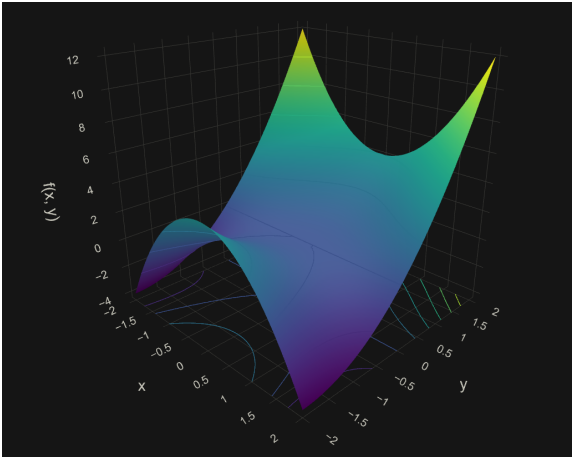
$f(x) = x^2, x_0 = 1, \alpha = 0.25$

Example 2

$$f(x, y) = x^2 y + y^2, \begin{bmatrix} x_0 \\ y_0 \end{bmatrix} = \begin{bmatrix} 2 \\ 4 \end{bmatrix}, \alpha = 1$$

Example 2 has no minimum

Gradient Descent happily runs off to $-\infty$



Your tasks

Do one step of gradient descent for the function

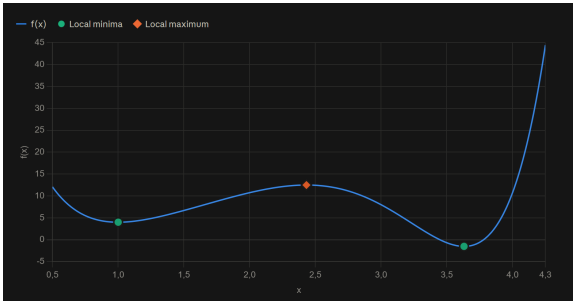
$$f(x) = 3x^2 - 1.25(x-1)^4 + (x-2)^6$$

with

1. $x_0 = 2, \alpha = 0.1$
2. $x_0 = 3, \alpha = 0.01$
3. $x_0 = 3, \alpha = 0.1$

Problem: local minima

Solution: candidate models (differ weight initialization, learning rate α, ...)



Feedback

Please use the text field



https://lgms.nl/p/presentation-feedback/?code=2652d4